

Agent Governance Protocol (AGP)

A Deterministic Governance Layer for Multi-Agent Systems

Public Review Draft 0.9.1

Author: Demian Alejandro Goldberg

Role: Independent researcher; protocol editor and project founder

Publication date: 20 July 2026

Document status: Experimental public review draft

Document license: To be declared before public release

Project: <https://agpprotocol.org>

Repository: <https://github.com/agpprotocol/agp>

Experimental research document. This draft is not a production standard, has not received independent security review, and does not claim standards-body endorsement. The companion normative specification remains under development.

Abstract

Multi-agent systems increasingly combine autonomous components built by different teams, frameworks, vendors, and organizations. Existing interoperability and workflow technologies can connect agents, expose tools, exchange messages, and coordinate tasks. They do not, by themselves, provide a portable method for proving who was authorized to decide, which evidence version was reviewed, how objections and vetoes were resolved, or whether a coordinator altered the decision history.

The Agent Governance Protocol (AGP) is an experimental governance layer for high-impact multi-agent decisions. AGP defines canonical governance inputs, signed authority actions, deterministic resolution rules, revocation-aware verification, and an append-only transparency history that can be replayed by an independent auditor. It is designed to complement, rather than replace, communication protocols, workflow engines, identity systems, or distributed consensus.

The current prototype includes independent Python and Go implementations. Its experimental conformance suites report 260/260 deterministic vectors, 10/10 signed-envelope vectors, and 8/8 transparency vectors passing with byte-identical receipts across implementations. A small adversarial benchmark detects eight of eight tested history and authority manipulations, while a deliberately minimal coordinator-trusting workflow baseline detects none. These results establish implementation consistency within the prototype; they do not establish production readiness, universal superiority over workflow systems, or external validation.

This public review draft states the problem, architecture, threat model, protocol objects, deterministic resolution model, validation evidence, limitations, and research agenda. Its central hypothesis is that systems making consequential multi-agent decisions may require a governance layer that is distinct from communication and orchestration, and independently verifiable by parties that do not fully trust the coordinator.

1. Executive Summary

AI systems are moving from isolated model calls toward networks of specialized agents. One agent may gather evidence, another may assess security, another may represent legal constraints, and another may authorize execution. The decision itself can be more consequential than any individual message: deploy software, approve a financial action, release data, alter policy, or trigger a physical process.

Most agent infrastructure focuses on two questions: how agents communicate and how work is executed. AGP focuses on a third question: how a collective decision can be governed and later verified.

AGP models a decision as a closed, canonical input set. That set includes the proposal, evidence manifest, authority snapshot, signed ballots, objections, vetoes, revocations, and resolution policy. Independent implementations resolve the same input into the same receipt. The receipt is appended to a hash-linked history. An auditor can reconstruct the decision without accepting the coordinator's database as authoritative.

The design assumes that a coordinator may be operationally necessary but should not be the sole source of truth. This assumption leads to five core properties:

Deterministic resolution: identical canonical inputs produce identical outputs.

Explicit authority: membership, roles, weights, veto rights, and key validity are decision inputs.

Evidence binding: a decision is bound to a specific evidence version and input root.

Signed actions: ballots and governance actions are attributable to cryptographic keys.

Transparent history: omission, replacement, reordering, truncation, and equivocation can be detected within the defined threat model.

AGP is not a blockchain, not a workflow engine, not a transport protocol, and not a truth oracle. It does not determine whether evidence is factually correct. It determines which evidence was governed, which authorized parties acted, and whether the declared rules produce the recorded outcome.

The immediate purpose of AGP is experimental: make the governance hypothesis precise enough to implement, challenge, reproduce, and reject or improve in public.

2. The Governance Gap

A multi-agent system can successfully exchange messages and still fail to provide trustworthy governance. Consider a deployment involving four authorities: engineering, security, legal, and operations. A coordinator receives their assessments and records an approval. Later, an incident occurs. The organization asks:

Was security's negative ballot omitted?

Was the legal veto submitted before the deadline?

Did the evidence package change after review?

Was the approving key already revoked?

Was the recorded quorum calculated against the correct membership snapshot?

Can an external auditor reproduce the outcome without trusting the coordinator?

A conventional workflow can be extended to answer these questions, but the answers are usually implementation-specific. Each organization must design its own event model, signing rules, canonicalization, revocation semantics, replay protection, history integrity, and audit receipts. The absence of a common governance layer creates fragmented controls and makes interoperability difficult.

The governance gap is not simply authorization. Authorization answers whether an actor may perform an action. Governance addresses how multiple authorized actors collectively produce a decision under declared rules, evidence, time bounds, objections, replacement semantics, and audit requirements.

The gap is also not identical to consensus. Distributed consensus protocols typically establish agreement on ordered state among replicas under a network and fault model. AGP begins from a different problem: a bounded governance decision among named authorities, where roles may be unequal, vetoes may be legitimate, evidence versions matter, and a portable audit receipt is required. AGP could use a consensus system as storage, but consensus is not required by the current protocol model.

3. Relationship to Existing Layers

AGP is intended to complement established and emerging layers.

Model Context Protocol (MCP) standardizes how AI applications connect to external systems, tools, resources, and context. Its official architecture describes a host-client-server model focused on context exchange and tool interaction [1][2]. AGP does not replace this function. An AGP-governed decision may reference evidence obtained through MCP, or authorize a tool invocation exposed through MCP.

Agent2Agent (A2A) standardizes communication and interoperability among independent agents, including discovery, task exchange, and interaction models [3][4]. AGP does not define how agents discover or communicate with each other. A2A messages could carry AGP proposals or ballots, but AGP is concerned with governance semantics rather than transport.

Business Process Model and Notation (BPMN) provides a standard graphical notation for business processes and executable process semantics [5]. Workflow engines coordinate activities, retries, dependencies, timers, and state transitions. AGP does not attempt to replace orchestration. It defines an auditable decision object that a workflow may invoke before performing a consequential action.

Identity and access management systems authenticate principals and enforce permissions. AGP relies on identity and key management but adds collective decision semantics: membership snapshots, roles, weighted ballots, objections, vetoes, evidence binding, and deterministic receipts.

Transparency logs make history tamper-evident. AGP incorporates a minimal hash-linked transparency profile for governance events, but does not attempt to be a general-purpose public log infrastructure.

The proposed layering is therefore:

Communication -> Coordination -> Governance -> Execution -> Audit

These layers may overlap in implementations, but separating their responsibilities improves portability and allows governance to be evaluated independently of a particular agent framework or workflow product.

3.1 Why not combine existing components?

A capable engineering team can assemble many AGP-like properties from existing components: an identity provider, a policy engine such as Open Policy Agent, signed attestations, event sourcing, an immutable database, a transparency log, and custom workflow logic. AGP does not claim that these components are individually unavailable. Its hypothesis is that consequential multi-party decisions need a shared interoperability object and common semantics across implementations.

Without a protocol, independently assembled systems tend to differ on canonicalization, authority snapshots, evidence binding, ballot replacement, veto precedence, revocation timing, receipt structure, extension handling, and replay behavior. Those differences matter when decisions cross organizational or product boundaries. AGP therefore attempts to standardize the portable governance package, deterministic resolver behavior, and audit receipt rather than replace the underlying components.

The value proposition must be tested empirically. AGP is justified only if adopting the common model is simpler, more portable, or more independently verifiable than building and maintaining equivalent controls separately. A future benchmark must compare AGP not only with a minimal workflow, but also with hardened workflows that incorporate signatures, policy evaluation, append-only storage, and independent verification.

4. Design Goals and Non-Goals

4.1 Design goals

Portable. A governance decision should not depend on one vendor, agent framework, database, or programming language.

Deterministic. Independent conforming implementations should produce the same resolution and receipt from the same canonical input.

Explicit. Authority, policy, evidence, timing, objections, vetoes, and revocation state should be represented rather than inferred from coordinator behavior.

Auditable. A third party should be able to verify the decision without trusting mutable application state.

Composable. AGP should be usable alongside MCP, A2A, workflow engines, IAM, databases, and message buses.

Fail-closed for defined integrity failures. Invalid signatures, revoked keys, stale evidence, conflicting ballots, and broken history should not silently produce a valid decision.

Minimal core, extensible profiles. The base model should remain small while allowing domain-specific policies.

4.2 Non-goals

AGP does not determine factual truth. Correct signatures over false evidence remain false evidence.

AGP does not replace transport, discovery, task delegation, workflow execution, IAM, key custody, or distributed consensus.

AGP does not guarantee availability. A malicious participant may refuse to vote or withhold evidence.

AGP does not eliminate collusion. A sufficient coalition of authorized participants can approve a harmful decision.

AGP does not provide confidentiality in the current prototype. Governance objects may contain sensitive metadata and require an external confidentiality layer.

AGP does not claim production maturity, legal enforceability, or standards-body endorsement.

5. System Model and Terminology

Proposal. A request for a governed outcome. It includes a unique identifier, action, scope, policy reference, evidence root, and validity window.

Evidence manifest. A canonical list of evidence objects or hashes reviewed for the decision. AGP binds governance to evidence identity and version, not to factual correctness.

Authority snapshot. The closed set of members eligible for the decision, including identifiers, roles, weights, veto capabilities, key identifiers, and validity state.

Ballot. A signed authority action such as approve, reject, abstain, or domain-specific choice.

Objection. A structured challenge to the proposal or evidence. A policy may classify objections as blocking or non-blocking.

Veto. A policy-defined authority action that prevents approval when valid.

Revocation. A declaration that a key or authority is no longer valid from a defined point or sequence.

Input root. A cryptographic digest over the canonical governance input. It prevents the coordinator from presenting different decision inputs under the same proposal identity without detection.

Resolution policy. Deterministic rules for quorum, weights, ties, vetoes, objections, replacement, deadlines, and outcome mapping.

Resolution receipt. The deterministic output containing the result, counted and excluded actions, policy evaluation, input root, and audit-relevant reasons.

Transparency event. An append-only event linked to the previous event hash.

Audit receipt. The output of replaying and validating the governance history.

Coordinator. The component that gathers inputs and invokes resolution. It is operationally trusted for availability but not fully trusted for integrity.

Auditor. A party that independently validates signatures, canonical inputs, resolution, and history.

6. Architecture and Decision Lifecycle

The reference lifecycle contains seven stages.

Stage 1 - Proposal creation. A proposer creates a bounded proposal with a stable identifier, declared action, policy, deadline, and evidence reference.

Stage 2 - Evidence closure. The coordinator or evidence service produces a canonical evidence manifest. Any material change creates a new evidence version and therefore a new input root.

Stage 3 - Authority closure. The authority snapshot is fixed for the decision. Later membership changes do not silently alter the denominator or veto set.

Stage 4 - Signed participation. Authorities submit signed ballots, objections, vetoes, or replacements. Envelopes carry key identifiers, sequence or nonce data, and expiration information.

Stage 5 - Deterministic resolution. A resolver validates the closed input and applies the referenced policy. The coordinator cannot choose among multiple valid interpretations.

Stage 6 - Transparency append. Proposal, evidence root, authority snapshot, accepted governance actions, and receipt are appended to a hash-linked history.

Stage 7 - Independent audit. An auditor retrieves the governance package, verifies history and signatures, reconstructs the canonical input, and reruns resolution.

The architecture deliberately separates resolver logic from coordinator storage. A conforming implementation may combine them operationally, but auditability depends on the ability to export the complete canonical package and reproduce the result elsewhere.

7. Canonical Data and Cryptographic Binding

Determinism begins with bytes. Semantically equivalent objects can serialize differently because of key ordering, whitespace, number formatting, Unicode normalization, or implementation-specific behavior. AGP therefore requires a declared canonical serialization profile.

The current prototype uses canonical JSON principles influenced by the JSON Canonicalization Scheme, RFC 8785 [6]. A future normative version should explicitly define supported data types, Unicode handling, numeric constraints, duplicate-key rejection, and treatment of unknown fields.

Conceptually:

```
canonical_input = Canonicalize(proposal, evidence_manifest, authority_snapshot, actions, policy)
```

```
input_root = SHA-256(canonical_input)
```

Each signed governance action binds to the proposal identifier and input root. A ballot valid for one evidence package must not be silently reusable for another.

Signatures in the current prototype use Ed25519, standardized in RFC 8032 [7]. The protocol should remain algorithm-agile by identifying the signature suite in each envelope, while conformance profiles may mandate a specific suite.

A verifier must reject, according to profile rules:

- malformed canonical objects;

- duplicate object keys;

- unsupported numeric representations;

- invalid signatures;

- unknown or not-yet-valid keys;

- expired envelopes;

- revoked keys at the relevant decision point;

- replayed envelope identifiers or nonces;

- mismatched proposal identifiers or input roots.

8. Deterministic Resolution Model

AGP separates semantic validity from policy resolution.

First, the verifier determines which submitted actions are structurally and cryptographically valid. Second, the resolver applies the policy to the valid action set.

Let M be the authority snapshot. Each member m has an optional weight $w(m)$, role set $r(m)$, and veto capability $v(m)$. Let B be the set of valid, non-superseded ballots. A basic weighted approval policy may define:

$\text{eligible_weight} = \text{sum of weights for members eligible under the policy}$
 $\text{participating_weight} = \text{sum of weights represented by counted ballots}$
 $\text{approval_weight} = \text{sum of weights for counted approval ballots}$
 $\text{quorum_met} = \text{participating_weight} / \text{eligible_weight} \geq \text{quorum_threshold}$
 $\text{approval_met} = \text{approval_weight} / \text{counted_decisive_weight} \geq \text{approval_threshold}$

A decision is approved only if quorum_met and approval_met are true, no valid veto is present, and no blocking objection remains unresolved.

The exact denominator must be explicit. Possible denominators include all eligible weight, participating decisive weight, or non-abstaining weight. Ambiguity here creates incompatible implementations.

Replacement semantics must also be deterministic. The current experimental model permits a later valid ballot by the same authority to replace an earlier ballot under a defined sequence rule. Two conflicting actions with the same sequence or replacement rank constitute equivocation and must be surfaced rather than resolved by arrival order.

Tie behavior must be defined by policy. A tie may reject, defer, escalate, or invoke a designated tie-break authority. It must not depend on map iteration order, database retrieval order, or coordinator preference.

The resolution receipt should include at least:

- protocol and policy version;
- proposal identifier;
- input root;
- final outcome;
- quorum calculation;
- approval calculation;
- counted actions;
- excluded actions and reasons;
- veto and objections;
- evidence version;
- authority snapshot identifier;
- deterministic receipt hash.

9. Signed Envelopes, Replay, Expiration, and Revocation

A signed envelope carries a typed governance payload and verification metadata. The minimal conceptual fields are:

envelope_id;
 protocol_version;
 payload_type;
 payload;
 signer_id;
 key_id;
 signature_suite;
 issued_at;
 expires_at;
 nonce or sequence;
 signature.

The signature covers the canonical envelope excluding the signature field itself.

Replay protection can be implemented with unique envelope identifiers, nonces, monotonically increasing sequences, or domain-specific replay stores. The conformance profile must state which mechanism is required and the retention period for replay state.

Expiration limits the useful lifetime of captured envelopes. Clock handling requires explicit tolerance and a declared time source. Systems with weak clock trust may use transparency-log sequence positions or externally witnessed timestamps.

Revocation is evaluated against the relevant governance point. A key revoked before a ballot is issued must be rejected. A key revoked after a valid ballot may remain valid for historical audit, depending on policy. The protocol must distinguish retrospective invalidation from prospective revocation.

Key rotation should preserve signer continuity without allowing an old key to authorize new actions. Authority identity and key identity are therefore separate fields.

10. Transparency and Independent Audit

The experimental transparency profile represents governance history as a sequence of canonical events. Each event contains a previous-event hash and its own event hash:

$\text{event_hash}[i] = \text{SHA-256}(\text{Canonicalize}(\text{event_without_hash}[i]) \parallel \text{event_hash}[i-1])$

The genesis event uses a profile-defined null or fixed previous hash.

This construction can detect modification, deletion from the middle, and reordering when the auditor possesses or trusts an expected head. It does not, by itself, prevent a log operator from presenting two internally valid forks to different auditors. Fork detection requires gossip, witnesses, external anchoring, or a stronger transparency service.

An audit procedure should:

- verify the hash chain and expected head;
- validate event schemas and sequence rules;
- reconstruct the proposal, evidence manifest, authority snapshot, and action set;

verify signatures, expiration, replay state, and revocation;
 reconstruct the canonical input and input root;
 rerun deterministic resolution;
 compare the computed and recorded receipts;
 emit an audit receipt with explicit failures.

The audit result is distinct from the governance outcome. A proposal may have a recorded outcome of approved while the audit result is invalid because the history is inconsistent or incomplete. This distinction prevents an application from treating a positive business outcome as proof of governance integrity.

11. Guarantee Boundaries and Availability Assumptions

AGP's strongest guarantees apply to governance inputs that are observed, signed, committed, and made available to the verifier. It provides integrity and reproducibility over that committed view. It does not, by itself, guarantee that every relevant action or fact becomes visible.

A malicious coordinator may withhold a ballot before it is published, suppress evidence before closure, or present different uncommitted views to different parties. A local hash chain cannot expose a split view unless at least one checkpoint, log head, witness signature, or conflicting package is observed outside the coordinator's control.

Accordingly, the base threat model distinguishes four properties:

Integrity: committed objects cannot be altered without invalidating hashes or signatures.

Attribution: signed actions can be associated with declared keys under the active trust model.

Reproducibility: conforming resolvers can derive the same receipt from the same canonical package.

Visibility: relevant objects reach auditors or witnesses. Visibility requires deployment mechanisms beyond the resolver itself.

AGP does not claim global completeness in the absence of an external publication or witness mechanism. Deployments requiring fork detection SHOULD distribute signed checkpoints to independent witnesses or use a mature transparency service. Deployments requiring availability SHOULD replicate governance packages independently of the coordinator.

12. Threat Model

11.1 Assets

The protected assets are governance meaning and provenance: the proposal, evidence identity, authority set, governance actions, resolution, and historical sequence.

11.2 Adversaries

Malicious coordinator. Omits a rejection, replaces evidence, changes authority membership, reorders events, or fabricates a receipt.

Compromised authority key. Submits unauthorized actions until revoked.

Storage attacker. Deletes, modifies, truncates, or forks governance history.

Network attacker. Replays or delays valid envelopes.

Malicious participant. Equivocates, submits conflicting actions, or exploits ambiguous policy language.

Colluding majority. Uses legitimate authority to approve a harmful decision.

11.3 Mitigations in the current prototype

Canonical input roots detect replacement of governed inputs. Signatures attribute actions. Replay and expiration checks limit reuse. Revocation removes invalid keys. Deterministic resolution prevents coordinator-selected interpretation. Hash-linked history exposes many forms of mutation. Independent implementations reduce the risk of one language-specific interpretation.

11.4 Residual risks

Endpoint compromise before signing remains outside protocol detection. False evidence can be correctly signed and governed. A sufficient authorized coalition can approve harmful outcomes. A private fork may remain undetected without witnesses. Denial-of-service can prevent quorum. Metadata may leak sensitive relationships. Cryptographic security depends on key custody and library correctness.

13. Conformance and Interoperability

A protocol is only portable if independently written implementations agree on edge cases. AGP therefore treats conformance vectors as a central artifact rather than optional tests.

The experimental suite contains semantic vectors for majority approval, vetoes, ties, blocking objections, revoked votes, stale evidence, equivocation, replacements, review revocation, non-members, and generated fuzz cases. Python and Go implementations reportedly produce byte-identical receipts for 260/260 vectors.

The signed-envelope suite covers valid signatures, payload tampering, signature tampering, replay, expiration, unknown keys, revoked keys, and related validation states. The current result is 10/10 vectors passing with byte-identical verification receipts.

The transparency suite covers clean history and selected mutations such as omission, alteration, reordering, truncation, and root replacement. The current result is 8/8 vectors passing with byte-identical audit receipts.

These results demonstrate internal interoperability between two implementations maintained within the same project. Stronger evidence requires:

an implementation written by an independent party;

published normative schemas;

randomized differential testing;

property-based and mutation testing;

test vectors reviewed by external security engineers;

compatibility across version changes.

14. Experimental Benchmark

The project includes a small comparison between an AGP verification path and a conventional workflow baseline. The scenarios include a clean approval plus eight adversarial cases: omitted ballot, altered ballot, reordered history, truncated history, stale evidence, revoked voter, replaced decision root, and duplicate ballot.

Reported result:

Workflow attack detection: 0/8 AGP attack detection: 8/8

A local macOS run reported approximately 56.8 ms for the workflow baseline and 59.1 ms for AGP in that test environment. This difference is not a general performance result; it describes one small implementation and workload.

The benchmark's most important limitation is structural: the workflow baseline intentionally trusts coordinator state and does not include custom signature, revocation, canonicalization, or transparency controls. A mature workflow system can be extended with such controls. Therefore the benchmark demonstrates the additional properties of AGP's explicit verification model, not that all workflow engines are insecure or that AGP is universally more efficient.

A credible next benchmark should compare:

AGP against a basic coordinator-trusting workflow;

AGP against a workflow with signatures and event logging;

AGP against a hardened workflow with immutable storage, policy evaluation, and independent verification;

implementation effort and lines of policy code;

latency at increasing authority and evidence counts;

storage growth and audit cost;

failure recovery and operational complexity;

usability for implementers and auditors.

15. Example: Governed Production Deployment

A software release is proposed for production. The policy requires engineering approval, security approval, operations quorum, and no legal veto.

The proposal references release artifact hash R, vulnerability scan S, test report T, and change plan C. These form evidence manifest E1. The authority snapshot contains five members with defined roles and keys.

Engineering approves. Operations approves. Security rejects because scan S contains an unresolved critical finding. The coordinator attempts to omit the security ballot and records approval.

Under a coordinator-trusting workflow, the database may simply show the recorded approval. Under AGP, the independent auditor receives the signed actions and transparency history. If the security ballot was included and then deleted, the hash chain or expected log head fails. If the coordinator constructs a new history without the ballot, external witnessing or a previously observed head is

needed to expose the fork. If the ballot is present but excluded from resolution, deterministic replay produces a different receipt.

The example illustrates both value and limits. AGP can make manipulation detectable when the relevant signed actions and history commitments are available. It cannot guarantee that every auditor learns about an action that was withheld before any shared commitment or witness observed it.

16. Deployment Profiles

AGP should not require one operational topology. Three illustrative profiles are possible.

Embedded profile. A single application embeds the resolver and exports signed governance packages. Appropriate for internal systems where the primary goal is deterministic audit.

Service profile. A dedicated governance service receives proposals and actions through an API, resolves decisions, and exposes audit packages. This improves reuse but creates an availability dependency.

Witnessed profile. Multiple organizations or witnesses observe log heads or co-sign checkpoints. This improves fork detection for cross-organization governance.

Profiles may also vary by confidentiality. A full-disclosure profile stores complete canonical objects. A hash-only profile stores commitments and releases sensitive evidence only to authorized auditors. Zero-knowledge or selective-disclosure approaches are future research and are not part of the current prototype.

17. Versioning and Evolution

Protocol evolution can itself undermine determinism if implementations silently interpret the same object under different rules. Every proposal, policy, envelope, and receipt must therefore identify applicable versions.

A compatible minor revision may add optional fields that older implementations safely ignore under declared extension rules. A breaking revision changes canonicalization, required fields, validation, or resolution semantics and requires a new major version or profile identifier.

Unknown critical extensions must cause rejection. Unknown non-critical extensions may be preserved and ignored if they are included consistently in canonicalization rules.

Conformance vectors should be versioned and immutable after publication. Corrections should create new vectors with an erratum record rather than rewriting historical expected outputs.

18. Governance of AGP Itself

A governance protocol should not depend indefinitely on unilateral control by its original author. However, premature foundation-building can create process without users.

During the experimental phase, the project should use transparent repository governance: public issues, design proposals, recorded rationale, versioned releases, and explicit maintainer decisions. Material protocol changes should include conformance vectors and compatibility analysis.

If external adoption emerges, the project should transition toward a multi-maintainer technical steering model. Trademark and certification rules should be separated from the open specification. Conformance claims should be evidence-based and reproducible.

The name AGP should identify compatibility with the public specification, not ownership of implementations. Commercial products may build on AGP while the core protocol remains openly implementable.

19. Limitations and Open Questions

The current work has important limitations.

No independent implementation. Both reference implementations were developed within the same project context.

No production deployment. The protocol has not been tested under real organizational latency, key rotation, partial evidence access, or operational failure.

Internal benchmark design. The adversarial benchmark was created by the protocol authors and may favor the protocol's abstractions.

Whitepaper/specification separation. This document explains the motivation, architecture, guarantees, and evidence. It is not the complete normative AGP Core Specification. Independent implementation will require versioned schemas, canonicalization rules, state-transition semantics, error codes, extension behavior, and test vectors in a separate specification.

Limited transparency model. A simple hash chain does not fully solve split-view attacks.

No confidentiality profile. Governance metadata may be sensitive.

No formal proof. Determinism is empirically tested, not mathematically proven for all implementations.

No usability evidence. It is unknown whether developers and auditors find AGP simpler than implementing equivalent controls directly.

Open questions include:

Which decisions justify AGP's complexity?

What is the minimal useful core?

How should evidence truth and provenance integrate with governance?

Can authority privacy coexist with public auditability?

What witness model is sufficient for practical fork detection?

How should human and AI authorities be distinguished, if at all?

Can governance policy be safely programmable without recreating smart-contract complexity?

20. Research and Adoption Roadmap

Phase A - Public review

Publish this draft, invite counterexamples, clarify normative semantics, and document disagreements. The success criterion is not positive feedback; it is discovery of ambiguous or unnecessary mechanisms.

Phase B - Implementer usability

Create a minimal SDK and a five-minute demonstration. Integrate with one agent framework and one workflow engine to prove complementarity rather than conceptual isolation.

Phase C - Independent reproduction

Recruit external implementers to reproduce receipts without using reference resolver code. Add a third language implementation and public interoperability runs.

Phase D - Real pilot

Use AGP for one bounded, reversible, high-value decision such as production deployment approval or controlled data release. Measure latency, operational burden, audit quality, and failure modes.

Phase E - Security review

Commission an external protocol and cryptography review. Expand the log model with witnesses or a mature transparency system if split-view resistance is required.

Phase F - Stable specification

Only after external implementation and pilot evidence, publish a stable 1.0 specification with normative schemas, compatibility policy, and conformance process.

21. Conclusion

Multi-agent systems need more than connectivity and orchestration when their collective decisions have material consequences. The missing property is not necessarily another messaging protocol or workflow language. It may be a portable governance object: a precise record of proposal, evidence, authority, signed participation, deterministic resolution, and auditable history.

AGP is an experimental attempt to define that object. The current prototype shows that two implementations can produce byte-identical governance and audit receipts across a substantial internal test suite, and that explicit integrity controls detect selected manipulations that a coordinator-trusting baseline does not.

Those results are promising but preliminary. The decisive tests are external: whether independent engineers can implement the specification, whether real systems find the distinction useful, whether the operational complexity is justified, and whether the design survives adversarial review.

AGP should therefore be evaluated as a falsifiable proposal, not promoted as an established standard. Its value will be determined by reproducibility, criticism, interoperability, and use.

References

- [1] Model Context Protocol, “Introduction,” official documentation, accessed 20 July 2026, <https://modelcontextprotocol.io/docs/getting-started/intro>
- [2] Model Context Protocol Specification, version 2025-11-25, official specification, <https://modelcontextprotocol.io/specification/2025-11-25>
- [3] Agent2Agent Protocol, “Agent2Agent Protocol Specification,” version 1.0 documentation, <https://a2a-protocol.org/latest/specification/>
- [4] Agent2Agent Protocol, “Announcing Version 1.0,” official project announcement, <https://a2a-protocol.org/latest/announcing-1.0/>
- [5] Object Management Group, Business Process Model and Notation (BPMN), Version 2.0.2, <https://www.omg.org/spec/BPMN/2.0.2/>

- [6] Open Policy Agent, official documentation, <https://openpolicyagent.org/docs>
- [7] M. Jones et al., “JSON Canonicalization Scheme (JCS),” RFC 8785, June 2020, <https://www.rfc-editor.org/rfc/rfc8785>
- [8] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032, January 2017, <https://www.rfc-editor.org/rfc/rfc8032>
- [9] S. Bradner, “Key words for use in RFCs to Indicate Requirement Levels,” RFC 2119, March 1997, <https://www.rfc-editor.org/rfc/rfc2119>
- [10] B. Leiba, “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words,” RFC 8174, May 2017, <https://www.rfc-editor.org/rfc/rfc8174>
- [11] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, December 2021, <https://www.rfc-editor.org/rfc/rfc9162>
- [12] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem,” ACM Transactions on Programming Languages and Systems, 4(3), 1982.
- [13] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” Proceedings of OSDI, 1999.
- [14] L. Lamport, “Paxos Made Simple,” ACM SIGACT News, 32(4), 2001.
- [15] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” USENIX ATC, 2014.
- [16] P. Hunt et al., “ZooKeeper: Wait-free Coordination for Internet-scale Systems,” USENIX ATC, 2010.
- [17] S. Haber and W. S. Stornetta, “How to Time-Stamp a Digital Document,” Journal of Cryptology, 3, 1991.
- [18] in-toto project, “A framework to secure the integrity of software supply chains,” official specification and project documentation, <https://in-toto.io/>
- [19] Supply-chain Levels for Software Artifacts (SLSA), official specification, <https://slsa.dev/spec/>
- [20] IETF, “About RFCs and document statuses,” <https://www.ietf.org/process/rfcs/>
- [21] AGP project repository, reference implementations and experimental suites, <https://github.com/agpprotocol/agp>

Citation and Review

Suggested citation:

Goldberg, Demian Alejandro. *Agent Governance Protocol (AGP): A Deterministic Governance Layer for Multi-Agent Systems*. Public Review Draft 0.9.1, 20 July 2026. <https://agpprotocol.org>

Technical feedback should be submitted through the public issue tracker in the AGP repository. Review is particularly requested on novelty, canonicalization, authority and revocation semantics, transparency assumptions, benchmark methodology, and the boundary between AGP and existing policy/workflow components.

Appendix A - Current Experimental Results

AGP v0.3 Conformance: 260/260 passed

AGP v0.4 Signed Conformance: 10/10 passed

AGP v0.5 Transparency: 8/8 passed

AGP vs Workflow benchmark: 8/8 attacks detected by AGP

These are project-reported, reproducible experimental results and should not be interpreted as independent certification.